
ACME 30 – Assignment no. 2: VPN on ACME co.

Alberto Pulcini 2046542
Nicolò Piovan 2219663
Giovanni Colasuonno 2046000
Davide Nardoni 2060903

June 2, 2026

1 Introduction

This report describes the work carried out by our group for the second assignment of the course. The activity was organised in the following phases:

1. **Initial brainstorming**, where we discussed the goals of the assignment and decided where to place the VPN gateway, how to structure the X.509 PKI and how to enforce the per-role access policy of the four road-warrior accounts.
2. **PKI and certificate setup**, in which we created the internal Certificate Authority and issued the server and client certificates used by both the OpenVPN service and the site-to-site IPSec tunnel.
3. **Road-warriors OpenVPN**, where we configured the OpenVPN server on the Main firewall, defined a dedicated tunnel subnet per role (Alice, Bob, Christina, Diana) and wrote the firewall rules that enforce the access matrix required by the assignment.
4. **Main-Internal IPSec tunnel**, in which we established an IPSec connection between the Main and the Internal firewalls so that the traffic crossing the transit link is authenticated and encrypted.
5. **IPv6 extension**, where we replicated the same OpenVPN and IPSec setup over IPv6 using a ULA pool for the tunnel addresses and the global prefixes delegated to the ACME-30 network.
6. **Tests**, used to validate that each road-warrior account can only reach the networks allowed by its role and that the site-to-site traffic actually flows inside the IPSec tunnel.
7. **Final remarks**, summarising the lessons learned and the main difficulties encountered.

The following sections detail each of these phases.

2 Brainstorming

The two main choices to make before touching any configuration were *where to terminate the road-warrior VPN* and *how to map the four user roles onto firewall rules*. The full topology is recalled in Figure 16.

2.1 Where to place the OpenVPN gateway

Per-role access (Alice → external services only, Diana → any net, etc.) can only be enforced where the firewall sees the decrypted tunnel traffic. We therefore terminate OpenVPN *on the Main firewall* itself, using OPNsense's built-in server: road warriors hit the public WAN IP, the tunnel is decrypted on a tun interface, and from that point on the firewall applies normal rules. A VPN box on the internal network or in the DMZ would either hide the user identity behind the encrypted UDP flow or force us to punch DMZ-to-internal holes, so both options were discarded.

2.2 Per-role addressing

The trick used to enforce the four roles is to give each user its own source range on the tunnel side, so that the firewall can write its rules in terms of subnets instead of usernames. We picked the 10.8.0.0/22 pool, split into four /24 blocks, one per role (Table 1). Each block is exported as a firewall alias (vpn_external, vpn_standard, vpn_power, vpn_admin) and used in four pass rules, closed by an explicit *any → any block* on the OpenVPN interface.

User	Role	Tunnel block
Alice	External	10.8.0.0/24
Bob	Standard	10.8.1.0/24
Christina	Power	10.8.2.0/24
Diana	Administrator	10.8.3.0/24

Table 1: One /24 per role inside the 10.8.0.0/22 pool.

2.3 PKI and IPSec choices

The X.509 authentication is built around an internal CA (RSA-2048) created on the Main firewall, which signs the OpenVPN server certificate, one client certificate per road warrior and the two firewall certificates used by the site-to-site tunnel. The same CA is reused for IPSec, so we avoid PSKs and can revoke a peer by revoking its certificate. For the IPSec Child SAs we use narrow selectors instead of any/any: only the Power and Admin VPN pools on the Main side and only the Internal server/client networks

on the other, so that a misconfigured OpenVPN rule cannot smuggle traffic from Alice or Bob through the tunnel.

3 PKI and certificate setup

All certificates were issued from a single internal Certificate Authority created on the Main firewall, with a 2048-bit RSA key and a 10-year validity. The CA was then used to sign three families of certificates:

- the *OpenVPN server* certificate, with Server key usage, bound to the WAN address of the Main firewall;
- one *client* certificate per road warrior (Alice, Bob, Christina, Diana), with the Common Name set to the username, so that individual revocation is possible;
- two *IPSec server* certificates, one per firewall, with the IP of the transit interface declared as Subject Alternative Name (100.100.254.1 for the Main and 100.100.254.2 for the Internal firewall), so that the peer can validate the address it is actually talking to.

Because the IPSec peer lives on a different appliance, the CA certificate and the Internal firewall certificate (with its private key) were exported as PKCS#12 from the Main firewall and re-imported on the Internal one. After this step both firewalls share the same trust anchor and each one holds the private key only for its own end of the tunnel.

In addition to the X.509 mutual authentication, an OpenVPN *TLS static key* was generated and embedded in the server configuration. It is a pre-shared HMAC that the client has to present in the very first packet: peers that do not carry the correct key are dropped silently, well before the TLS handshake starts, which makes handshake-level DoS attacks much harder to mount.

3.1 Cryptographic parameters

All keys and channels use the OPNsense defaults, which are already a reasonable modern baseline. The CA and every certificate it signs (OpenVPN server, road-warrior clients, IPSec server certificates) use **RSA-2048** as the key algorithm and **SHA-256** as the signature digest. The OpenVPN *TLS static key* is a 2048-bit OpenVPN static-key block (PEM V1 format), used in `tls-auth` mode as an HMAC envelope on the control channel.

For the data channel the OpenVPN server is configured with the default OPNsense cipher list `AES-256-GCM:AES-128-GCM:CHACHA20-POLY1305` and **SHA-256** as the auth digest [5]; the negotiation ends up on **AES-256-GCM**, an AEAD cipher that provides both confidentiality and integrity. The TLS control channel runs on TLS 1.2+ with the same RSA-2048 server certificate.

The IPSec connection uses IKEv2 with the OPNsense defaults too: IKE proposal **AES-256 + SHA-256 + DH group 14** (MODP-2048), and ESP proposal **AES-256-GCM** (128-bit ICV) with PFS on DH group 14. IKE SA lifetime is 28800 s,

ESP SA lifetime 3600 s, both with DPD 30/120 s as already mentioned.

4 Road-warriors OpenVPN

The OpenVPN server runs on the Main firewall, bound to the WAN address 100.100.0.2 on UDP 1194. It uses *topology subnet* over the 10.8.0.0/22 pool described in the brainstorming and authenticates clients with the internal CA plus the TLS static key (cryptographic parameters listed in Section 3).

4.1 Per-user overrides

Each user is bound to its own /30 link inside the pool through a *client-specific override* that pins the tunnel network and pushes only the routes the role is supposed to reach. The four overrides are summarised in Table 2.

User	Tunnel	Pushed routes
Alice	10.8.0.0/30	ext (.4)
Bob	10.8.1.0/30	DMZ (.6)
Christina	10.8.2.0/30	DMZ + servers (.1)
Diana	10.8.3.0/30	ext, DMZ, servers, clients

Table 2: Client-specific overrides. The third column lists the third octet of the 100.100.X.0/24 subnets pushed to the client.

Every override also carries the *push-reset* flag, which wipes any global push directive coming from the server config (including the default `redirect-gateway`) before the per-user routes are applied. This way each client receives *only* the explicit routes listed in its own override, and no traffic can leak through the tunnel by inheriting a server-wide push by mistake.

4.2 Firewall rules

A single rule on the WAN interface lets UDP 1194 reach the firewall itself; everything else stays closed. On the OpenVPN interface the access matrix is implemented as four pass rules using the four aliases defined in Section 2.2, followed by a final default-deny rule (Figure 1):

- `vpn_external` → external services net (pass)
- `vpn_standard` → DMZ net (pass)
- `vpn_power` → DMZ + internal servers (pass)
- `vpn_admin` → any (pass)
- any → any (block)

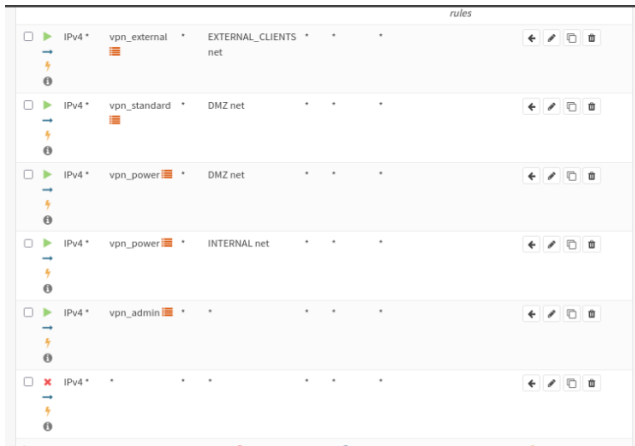


Figure 1. OpenVPN interface rules on the Main firewall.

4.3 Client connection

Each client receives an exported .ovpn bundle (configuration, CA, client certificate and TLS static key in a PKCS#12 file) and connects with the standard openvpn CLI. The handshake log for Alice is:

```
$ sudo openvpn ACME_30_Roadwarriors_alice.ovpn
TCP/UDP: Preserving recently used remote
address: [AF_INET]100.100.0.2:1194
[openvpn-server] Peer Connection Initiated
with [AF_INET]100.100.0.2:1194
DCO device tun0 opened
net_addr_v4_add: 10.8.0.2/22 dev tun0
Initialization Sequence Completed
```

After the connection tun0 carries the expected /30 end-point inside Alice's block and the kernel only installs the routes for the external services network, exactly as planned in the override.

4.4 Access matrix verification

To check that the firewall actually enforces the access matrix we ran a pair of pings from Alice's laptop through tun0: one toward 100.100.4.1 (external services, allowed) and one toward 100.100.2.1 (clients network, denied):

```
$ ping -I tun0 100.100.4.1
64 bytes from 100.100.4.1:
icmp_seq=1 ttl=64 time=25.7 ms
64 bytes from 100.100.4.1:
icmp_seq=2 ttl=64 time=25.7 ms
3 packets transmitted, 3 received,
0% packet loss

$ ping -I tun0 100.100.2.1
4 packets transmitted, 0 received,
100% packet loss
```

The first ping succeeds, the second one is silently dropped by the *any* → *any block* on the OpenVPN interface, confirming that Alice's role is correctly enforced. The same test was repeated for each of the other three accounts and in every case only the subnets allowed by the role answered the probes.

4.5 Encryption check with Wireshark

To make sure that the tunnel is actually doing its job we captured the same Alice ping toward 100.100.4.1 on two different interfaces: tap0, which is the simulated physical link to the Internet (where the OpenVPN-encapsulated UDP packets flow), and tun0, which is the in-kernel side of the decrypted tunnel.

On tap0, filtering with `udp.port == 1194`, Wireshark sees only OpenVPN UDP datagrams whose payload is opaque (Figure 2); no ICMP frame is ever recognised on this interface.

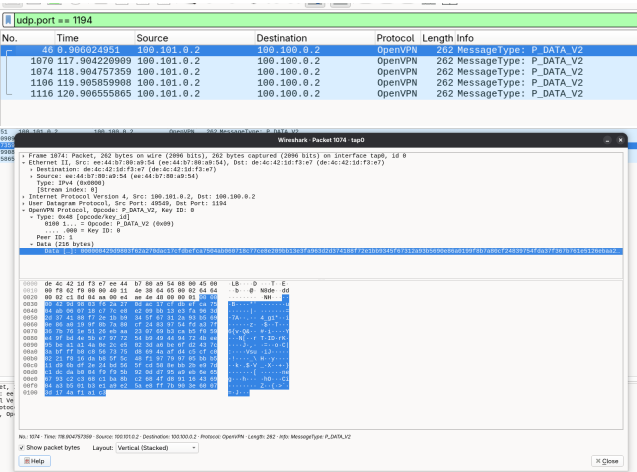


Figure 2. Capture on tap0 during Alice’s ping: only OpenVPN UDP 1194 datagrams with random-looking payload.

On tun0, instead, Wireshark sees the cleartext ICMP echo request/reply pair toward 100.100.4.1 with the expected payload (Figure 3), confirming that the firewall decrypts the traffic only on the tunnel side and that the simulated Internet only ever observes the encrypted OpenVPN flow.

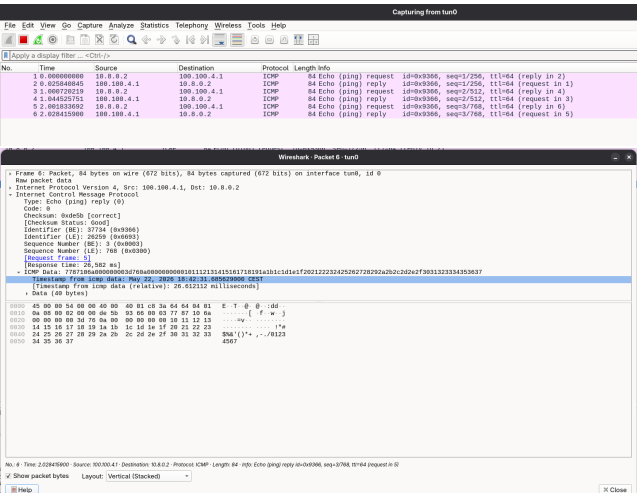


Figure 3. Same ping captured on tun0: cleartext ICMP echo request/reply with readable payload.

5 Main–Internal IPsec tunnel

The site-to-site tunnel is an IKEv2 IPsec connection between the 100.100.254.1 interface of the Main firewall and the 100.100.254.2 interface of the Internal one. Both peers authenticate each other with the certificates issued in Section 3¹ and run Dead Peer Detection with a 30 s delay and a 120 s timeout, so that a half-broken tunnel is renegotiated automatically. Cipher suites are the OPNsense defaults summarised in Section 3.

¹ Reusing the same CA created for OpenVPN.

5.1 Child SAs

The Child SAs use narrow traffic selectors instead of any/any, so that the tunnel only carries the traffic that the role-based policy allows to reach the Internal site:

- Main side: local 10.8.2.0/24, 10.8.3.0/24 (Power + Admin VPN pools); remote 100.100.1.0/24, 100.100.2.0/24 (Internal servers + Clients).
- Internal side: the same pairs, mirrored.

Even if a future bug in the OpenVPN ruleset let Alice or Bob send a packet toward an internal address, IPsec would refuse to encapsulate it, since the source would not match any selector.

5.2 Firewall rules

The Power pool is only allowed to reach the internal servers alias (internal_server_net ≡ 100.100.1.0/24), while the Admin pool can reach anything. On the Internal firewall the same two aliases (vpn_power, vpn_admin) are recreated and used to write the symmetric pass rules that let the internal hosts answer back. Table 3 reports the complete ruleset on the IPsec interface of both firewalls.

FW	Source	Destination
Main	vpn_power	internal_server_net
Main	vpn_admin	any
Internal	internal_server_net	vpn_power
Internal	vpn_power	internal_server_net
Internal	any	vpn_admin
Internal	vpn_admin	any

Table 3: IPsec pass rules on both firewalls. Everything else is dropped by the implicit default-deny.

5.3 Connection status

After applying the configuration on both peers the IKE Security Association and both Child SAs come up cleanly (Figure 4), and the OpenVPN dashboard shows the expected road-warrior sessions (Figure 5).

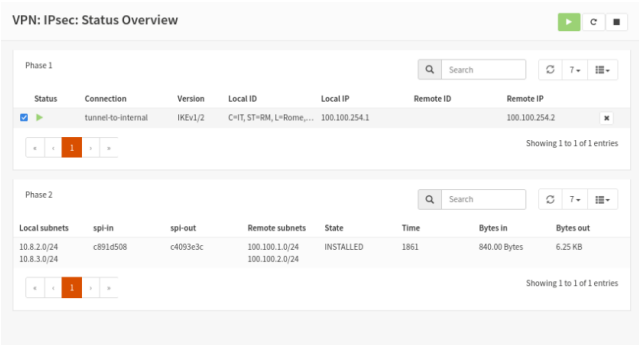


Figure 4. IPsec status overview on the Main firewall.

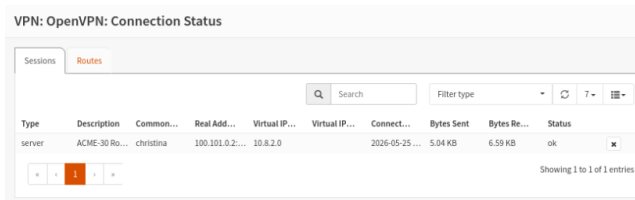


Figure 5. OpenVPN active sessions.

5.4 End-to-end test

To exercise both VPNs at once we logged in as Christina (Power) and pinged the DNS server inside the Internal servers network:

```
$ ping -I tun0 100.100.1.2
64 bytes from 100.100.1.2:
  icmp_seq=1 ttl=62 time=31.0 ms
64 bytes from 100.100.1.2:
  icmp_seq=2 ttl=62 time=30.0 ms
3 packets transmitted, 3 received,
0% packet loss
```

The packet path can be observed at three different points. On the laptop's tap0 interface only OpenVPN UDP 1194 traffic is visible (Figure 6): nothing reveals the real IPs of the internal server or of the firewall transit link.

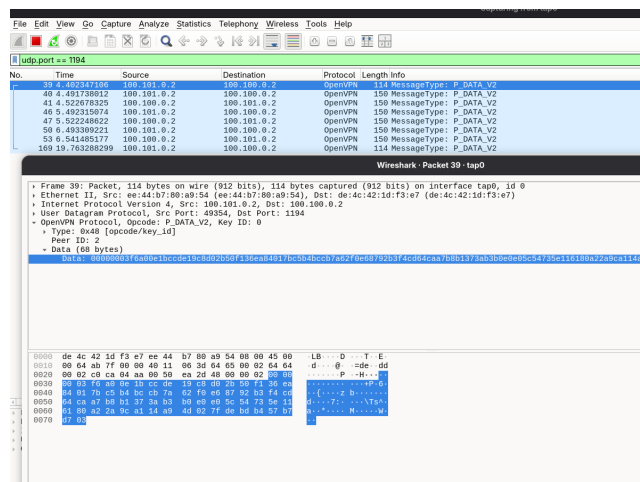


Figure 6. tap0 capture during Christina's ping: only the outer OpenVPN flow is visible.

After OpenVPN decryption on the Main firewall the packet enters the IPsec tunnel toward the Internal firewall. Capturing on the transit interface of the Main firewall we see only ESP (protocol 50) packets, with no readable inner header (Figure 7). Finally, on Christina's tun0 interface the original ICMP exchange is fully readable (Figure 8).



Figure 7. Capture on the Main firewall's transit interface: only ESP packets toward 100.100.254.2.

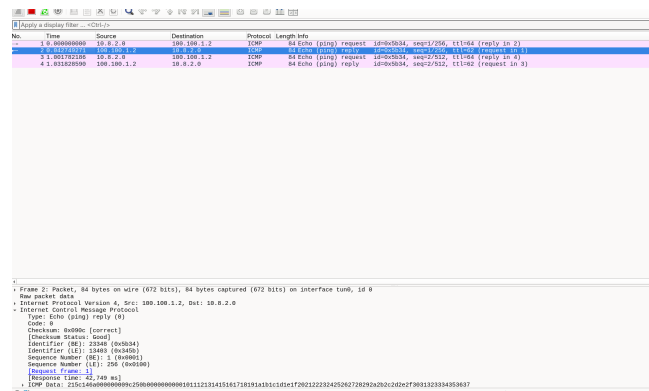


Figure 8. Same ping seen on Christina's tun0: cleartext ICMP.

A traceroute from Christina toward the same DNS server makes the behaviour of the two stacked tunnels explicit:

```
$ traceroute 100.100.1.2
1  10.8.0.1          32.4 ms
2  * * *
3  100.100.1.2      60.0 ms
```

Hop 1 is the OpenVPN gateway on the Main firewall; hop 2 is invisible because the packet is encapsulated in ESP and the intermediate router does not decrement the TTL on the tunnel interface; hop 3 is the final destination. The silent intermediate hop is the behavioural signature of the IPsec tunnel.

6 IPv6 extension

Since the ACME-30 network is dual-stack from Assignment 1, we replicated the whole VPN stack on IPv6 as well. The design mirrors the IPv4 one: ULA pool fd00:8::/64 split into four /112 blocks (one per role) for the OpenVPN tunnel, and the same IPsec connection extended with a second pair of selectors over the global IPv6 transit addresses.

6.1 OpenVPN IPv6

The OpenVPN server gets fd00:8::/64 as its IPv6 network, and each user receives a /112 link inside the pool through the same client-specific override mechanism used

on IPv4 (Table 4). The pushed local networks are the global 2001:470:b5b8:1eXX::/64 prefixes assigned to the corresponding IPv4 subnets in Assignment 1.

User	Tunnel	Pushed prefix
Alice	fd00:8::0:0/112	1e04::/64
Bob	fd00:8::1:0/112	1e06::/64
Christina	fd00:8::2:0/112	1e06::/64 + 1e81::/64
Diana	fd00:8::3:0/112	all

Table 4: Client-specific overrides on IPv6. Prefixes are abbreviated removing the common 2001:470:b5b8: part.

The four /112 blocks are exported as IPv6 firewall aliases (vpn_external_v6, vpn_standard_v6, vpn_power_v6, vpn_admin_v6), plus a dedicated internal_server_net_v6 for the 1e81::/64 prefix, and used in the same role-based pass rules as IPv4 followed by a final default-deny.

6.2 IPv4-encapsulated IPv6

The OpenVPN instance was configured with proto udp4 on UDP 1194, so the IPv6 tunnel traffic is carried inside IPv4 OpenVPN datagrams: on tap0 Wireshark sees IPv4 packets with an encrypted OpenVPN payload (Figure 9), while on tun0 the same packet appears as a raw IPv6 datagram with a cleartext ICMPv6 echo inside (Figure 10). This is not a problem for the security goal — the payload is still protected by OpenVPN — it only means that the outer transport is IPv4.

No.	Time	Source	Destination	Protocol	Length	Info
361	13.467582705	100.101.0.2	100.100.0.2	OpenVPN	170	MessageType: P_DATA_V2
362	13.509042882	100.100.0.2	100.101.0.2	OpenVPN	170	MessageType: P_DATA_V2
378	14.469322090	100.101.0.2	100.100.0.2	OpenVPN	170	MessageType: P_DATA_V2
380	14.499059256	100.100.0.2	100.101.0.2	OpenVPN	170	MessageType: P_DATA_V2
410	15.470976983	100.101.0.2	100.100.0.2	OpenVPN	170	MessageType: P_DATA_V2
413	15.637735099	100.100.0.2	100.101.0.2	OpenVPN	170	MessageType: P_DATA_V2

Figure 9. tap0: the IPv6 ping is carried inside IPv4 OpenVPN datagrams on UDP 1194.

No.	Time	Source	Destination	Protocol	Length	Info
3	0.000000000	fd00:8::0:0:0:0:0:0	fd00:8::1:0:0:0:0:0	ICMPv6	84	echo (ping) request id=0x0000, seq=1, hop limit=64 (reply in 3) [SendMTU=20, Round=21]
4	0.000000000	fd00:8::1:0:0:0:0:0	fd00:8::0:0:0:0:0:0	ICMPv6	84	echo (ping) reply id=0x0000, seq=1, hop limit=64 (request in 1) [SendMTU=20, Round=21]
5	0.000000000	fd00:8::0:0:0:0:0:0	fd00:8::1:0:0:0:0:0	ICMPv6	84	echo (ping) request id=0x0000, seq=2, hop limit=64 (reply in 4) [SendMTU=20, Round=21]
6	0.000000000	fd00:8::1:0:0:0:0:0	fd00:8::0:0:0:0:0:0	ICMPv6	84	echo (ping) reply id=0x0000, seq=2, hop limit=64 (request in 3) [SendMTU=20, Round=21]
7	0.000000000	fd00:8::0:0:0:0:0:0	fd00:8::1:0:0:0:0:0	ICMPv6	84	echo (ping) request id=0x0000, seq=3, hop limit=64 (reply in 6) [SendMTU=20, Round=21]
8	0.000000000	fd00:8::1:0:0:0:0:0	fd00:8::0:0:0:0:0:0	ICMPv6	84	echo (ping) reply id=0x0000, seq=3, hop limit=64 (request in 5) [SendMTU=20, Round=21]

Figure 10. tun0: same packet decapsulated, plain IPv6 ICMPv6 echo request.

6.3 Per-role test on IPv6

Connected with Alice's certificate, tun0 receives the expected fd00:8::/112 endpoint and an IPv6 route only toward the external services prefix:

```
inet6 fd00:8::/112 scope global
inet6 2001:470:b5b8:1e04::/64 dev tun0
metric 200 pref medium
```

A ping toward an external client succeeds, while the same probe toward the DMZ webserver returns Network is unreachable, because the override did not push that route. Forcing the route manually with `ip -6 route add 2001:470:b5b8:1e06::/64 dev tun0` the host generates the packet, but the firewall correctly drops it (100 % loss), confirming that the IPv6 rules enforce the access matrix independently of the client-side routing table.

6.4 IPsec IPv6

The site-to-site tunnel was extended by adding a second child to the existing IPsec connection, this time between the global IPv6 transit addresses (2001:470:b5b8:1e0f:d0cb:c3ff:fe2b:367e on the Main, 2001:470:b5b8:1e0f::2 on the Internal). The new selectors carry only the Power and Admin v6 pools toward the Internal v6 prefixes:

- Local: fd00:8::2:0/112, fd00:8::3:0/112.
- Remote: 2001:470:b5b8:1e81::/64, 2001:470:b5b8:1e82::/64.

The settings are mirrored on the Internal firewall and the dual-stack connection comes up cleanly (Figure 11). The corresponding IPsec firewall rules on both peers are shown in Figures 12–13.

Local subnets	spi-in	spi-out	Remote subnets	State	Time	Bytes in	Bytes out
100.100.1.0/24	c7f80c3	c7f15257	10.8.2.0/24	INSTALLED	110		
2001:470:b5b8:1e0f::2	c0e0fde	c0f8512	2001:470:b5b8:1e0f::2	INSTALLED	110		

Figure 11. IPsec status on the Internal firewall after adding the IPv6 child.

Protocol	Local	Remote	Action	Priority	State
IPv6 UDP	2001:470:b5b8:1e0f::2	*	This Firewall (ISAKMP)	500	*
IPv6 UDP	2001:470:b5b8:1e0f::2	*	This Firewall (IPsec NAT-T)	4500	*
IPv6 ESP	2001:470:b5b8:1e0f::2	*	*	*	*

Figure 12. IPsec firewall rules on the Main firewall.

Protocol	Local	Remote	Action	Priority	State
IPv6*	vpn_power_v6	internal_server_net_v6	*	*	*
IPv6*	vpn_admin_v6	*	*	*	*

Figure 13. IPsec firewall rules on the Internal firewall.

6.5 End-to-end IPv6 test

Connected with Christina's certificate, a ping toward the DNS server's global IPv6 succeeds end-to-end:

```
$ ping -6 -I tun0
2001:470:b5b8:1e81:4730:4341:85e8:4a85
64 bytes ... icmp_seq=1 time=123 ms
64 bytes ... icmp_seq=2 time=100 ms
64 bytes ... icmp_seq=3 time=108 ms
3 packets transmitted, 3 received,
0% packet loss
```

The same test was repeated with Diana's certificate, capturing the packet at three different points. On the laptop's tap0 only the encrypted OpenVPN flow is visible (Figure 14); on the Main firewall's transit interface only ESP packets are seen (Figure 15); on Diana's tun0 the original ICMPv6 echo is fully readable.

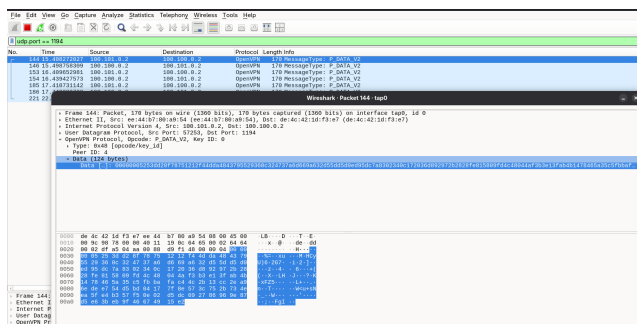


Figure 14. Diana's ping seen on tap0: encrypted OpenVPN UDP flow.

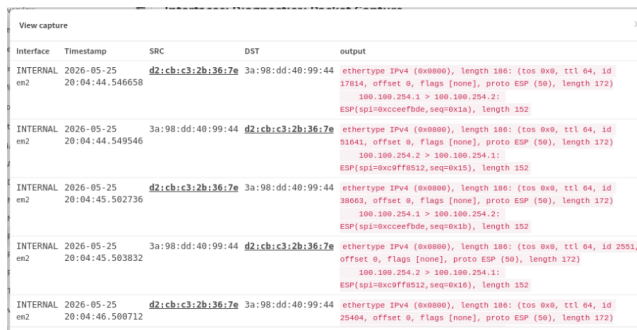


Figure 15. Same ping captured on the Main firewall's transit interface: only ESP packets.

The traceroute confirms the two stacked tunnels exactly as on IPv4:

```
$ traceroute -6
2001:470:b5b8:1e81:4730:4341:85e8:4a85
1 fd00:8::1 43.5 ms
2 * * *
3 2001:470:b5b8:1e81:... 43.4 ms
```

Hop 1 is the OpenVPN gateway on the Main firewall, hop 2 is the IPsec-encrypted transit (invisible to traceroute), hop 3 is the DNS server itself.

7 Final remarks

The two VPN services were brought up successfully on both address families. The road-warriors OpenVPN [5] terminates on the Main firewall and authenticates the four accounts through an internal X.509 PKI [3]; the per-user /30+/112 tunnel links translate directly into the four pass rules that implement the External/Standard/Power/Admin access matrix, and the wireshark captures confirm that nothing leaks in cleartext on the outer interface. The Main-Internal site-to-site link is protected by an IKEv2 [2] IPsec [1] tunnel that reuses the same CA, runs DPD on both peers and uses narrow Child-SA selectors as defence-in-depth against firewall-rule mistakes. The IPv6 extension reuses the same logic on a fd00:8::/64 ULA pool [4] and a second IPsec child between the global IPv6 transit addresses.

The setup is functional but could be hardened further on a real deployment: revocation today is local-only, so a CRL distribution point (or an OCSP responder) should be published so that revoked client certificates take effect across reboots; the OpenVPN instance is currently UDPv4-only (with IPv6 carried inside), and enabling proto udp6 would remove the IPv4 dependency for road warriors that have only IPv6 connectivity; finally, multi-factor authentication on top of the client certificate (e.g. static challenge or a TOTP plugin) would protect against the theft of a single .ovpn bundle.

References

- [1] S. Kent, K. Seo, *Security Architecture for the Internet Protocol*, RFC 4301, December 2005. <https://datatracker.ietf.org/doc/html/rfc4301>
- [2] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, T. Kivinen, *Internet Key Exchange Protocol Version 2 (IKEv2)*, RFC 7296, October 2014. <https://datatracker.ietf.org/doc/html/rfc7296>
- [3] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, W. Polk, *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, RFC 5280, May 2008. <https://datatracker.ietf.org/doc/html/rfc5280>
- [4] R. Hinden, B. Haberman, *Unique Local IPv6 Unicast Addresses*, RFC 4193, October 2005. <https://datatracker.ietf.org/doc/html/rfc4193>
- [5] OpenVPN Inc., *OpenVPN 2.6 Reference Manual*. <https://openvpn.net/community-resources/reference-manual-for-openvpn-2-6/>

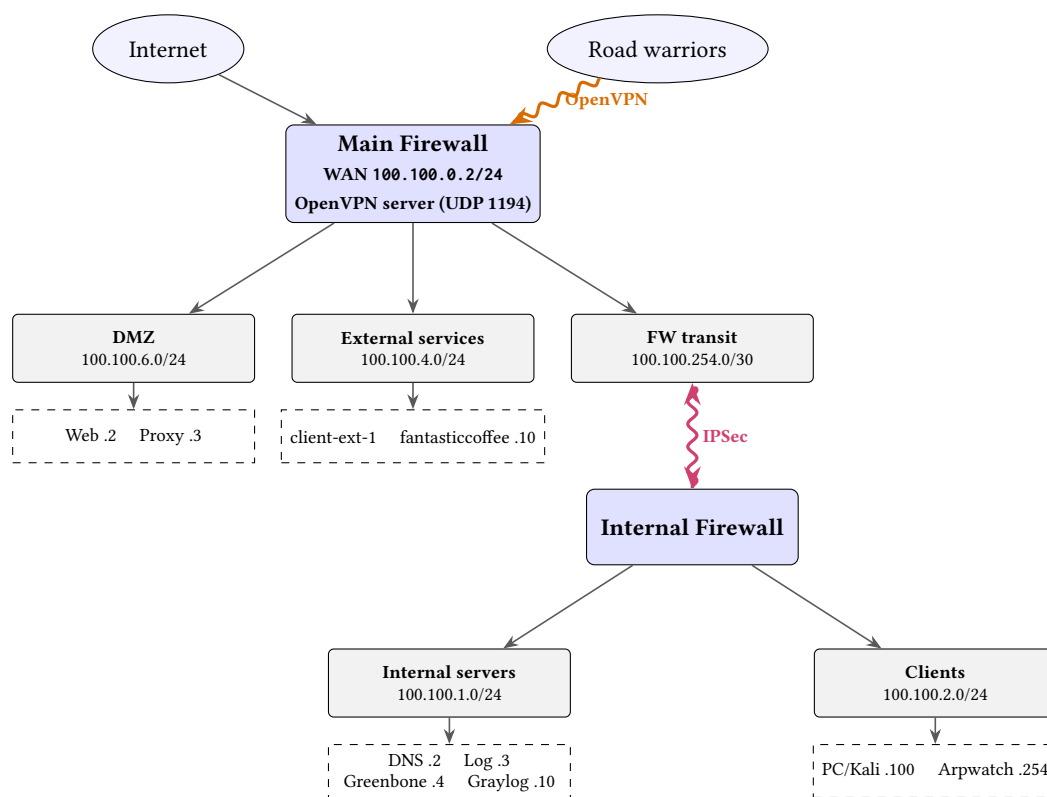


Figure 16. ACME-30 topology with the two VPN services introduced in this assignment: the road-warriors OpenVPN terminates on the WAN interface of the Main firewall, while the IPSec tunnel protects every packet flowing on the transit link between the Main and the Internal firewalls.